

Contents

1	Number Theory	1
1.1	Criba de Gries-Misra $\mathcal{O}(N)$	1
1.2	Discrete Log $\mathcal{O}(\sqrt{m})$	1
2	Estructuras de Datos	1
2.1	Fenwick Tree $\mathcal{O}(\log m)$	1
3	Grafos	1
3.1	Eulerian Path/Circuit	1
3.2	Max-Flow $\mathcal{O}(m^2 \log c)$	1
4	Strings	1
4.1	Z-algorithm $\mathcal{O}(n)$	1
4.2	Suffix Array	2

1 Number Theory

1.1 Criba de Gries-Misra $\mathcal{O}(N)$

```

1 vector<int> mindiv;
2 vector<int> primes;
3
4 void sieve(int N) {
5     mindiv = vector<int>(N+1, 0);
6     primes = vector<int>();
7     for(int i=2; i <= N; ++i) {
8         if(mindiv[i] == 0) {
9             mindiv[i] = i;
10            primes.push_back(i);
11        }
12        for(int j=0; j < primes.size() and
13            primes[j] <= mindiv[i] and primes[j]*i
14            <= N; ++j) {
15            mindiv[i*primes[j]] = primes[j];
16        }
17    }
18 }

```

1.2 Discrete Log $\mathcal{O}(\sqrt{m})$

Se basa en verificar si la ecuación en congruencias $a^x \equiv b \pmod{m}$ tiene solución y, en caso afirmativo, dar el valor de tal x . La idea clave es expresar $x = nq - p$, donde n nos interesará tomarlo lo más próximo posible a $\sqrt{m}$.

2 Estructuras de Datos

2.1 Fenwick Tree $\mathcal{O}(\log m)$

```

1 void update(int i) {
2     while (i <= m) ++st[i], i += i&-i;
3 }
4
5 int query(int i) {
6     int s = 0;
7
8     while (i >= 1) s += st[i], i -= i&-i;
9
10    return s;
11 }

```

3 Grafos

3.1 Eulerian Path/Circuit

```

1 stack<int> st; st.push(1);
2 vector<int> pth;
3 while (st.size()) {
4     int u = st.top();
5     bool f = false;
6     while (G[u].size()) {
7         int v = G[u].back().first, i = G[u]
8         ].back().second; G[u].pop_back();
9         if (!vis[i]) {
10            st.push(v);
11            vis[i] = true, f = true;
12            break;
13        }
14    }
15    if (!f) pth.push_back(u), st.pop();
16 }

```

3.2 Max-Flow $\mathcal{O}(m^2 \log c)$

```

1 bool dfs(int u, int mini, int k) {
2     if (vis[u]) return false;
3     if (u == n) {
4         val = mini;
5         return true;
6     }
7     vis[u] = true;
8
9     for (int v = 1; v <= n; ++v) if (G[u][v]
10    >= k) {
11         if (dfs(v, min(mini, G[u][v]), k))
12         {
13             G[u][v] -= val;
14             G[v][u] += val;
15             return true;
16         }
17     }
18     return false;
19 }
20
21 int ans = 0;
22 for (int k = 1e9; k; k >>= 1) {
23     vis = vector<bool>(n + 1, false);
24     while (dfs(1, 1e9 + 5, k)) {
25         ans += val;
26         vis = vector<bool>(n + 1, false);
27     }
28 }

```

4 Strings

4.1 Z-algorithm $\mathcal{O}(n)$

```

1 int n = s.size();
2 vector<int> z(n);
3 int x = 0, y = 0;
4
5 for (int i = 1; i < n; i++) {

```

```

6     z[i] = max(0, min(z[i - x], y - i + 1)
7 );
8     while (i + z[i] < n && s[z[i]] == s[i
9 + z[i]]) {
10         x = i, y = i + z[i], z[i]++;
11     }
12     if (i > m && z[i] == m) ans++;
13 }

```

4.2 Suffix Array

```

1 int n = s.size();
2
3 s += '$';
4
5 vector<iii> a;
6 for (int i = 0; i <= n; ++i) a.push_back
7     ({{s[i], 0}, i});
8 sort(a.begin(), a.end());
9
10 for (int k = 1; k <= n; k <= 1) {
11     vector<int> p(n + 1);
12
13     p[a[0].second] = 0;
14
15     for (int i = 1; i <= n; ++i) {

```

```

16         p[a[i].second] = p[a[i - 1].second
17 ];
18         if (a[i].first != a[i - 1].first)
19 ++p[a[i].second];
20     }
21
22     for (int i = 0; i <= n; ++i) a[i] = {{
23 p[i], p[(i + k) % (n + 1)]}, i};
24
25     sort(a.begin(), a.end());
26 }
27
28 vector<int> p(n + 1);
29 for (int i = 0; i <= n; ++i) p[a[i].second
30 ] = i;
31
32 int k = 0;
33 vector<int> lcp(n + 1, 0);
34 for (int i = 0; i < n; ++i) {
35     int j = a[p[i] - 1].second;
36
37     k = max(0, k - 1);
38
39     while (i + k < n && j + k < n && s[i +
40 k] == s[j + k]) ++k;
41
42     lcp[p[i]] = k;
43 }

```